

tipo Dizionario:

dati: un insieme S di coppie (*elem*, *chiave*)

operazioni:

insert(*elem e*, *chiave k*)

aggiunge a S una nuova coppia (e , k)

delete(*elem e*)

cancella da S l'elemento e

search(*chiave k*) $\rightarrow$ *elem*

se la chiave k è presente in S restituisce un elemento e ad essa associato, e null altrimenti

Figura 6.1 Il tipo di dato Dizionario.

di insiemi. Una soluzione alternativa al problema del dizionario, non basata su alberi, è fornita dalle tavole hash, e sarà discussa nel Capitolo 7.

6.1 Alberi binari di ricerca

Prima di definire il concetto di albero binario di ricerca, che è alla base di molte delle realizzazioni di dizionari che studieremo in questo capitolo, ricordiamo l'algoritmo di ricerca binaria in un array ordinato X che abbiamo introdotto nel Paragrafo 2.4.3 del Capitolo 2. Al primo passo, confrontando l'elemento da ricercare con l'elemento $X[n/2]$, ci era possibile restringere la ricerca nella metà inferiore o in quella superiore dell'array X . La procedura veniva poi applicata ricorsivamente. Immaginiamo ora di associare all'array ordinato X un albero binario T così costruito: la radice di T contiene il valore $X[n/2]$, mentre i valori più piccoli e più grandi di $X[n/2]$ sono assegnati rispettivamente ai nodi del sottoalbero sinistro e destro della radice di T , in modo ricorsivo. È facile vedere che una visita in ordine simmetrico dell'albero così ottenuto corrisponde ad una scansione di X dall'elemento più piccolo a quello più grande, e che ricercare in X un elemento corrisponde a tracciare un cammino dalla radice ad una foglia di T . La corrispondenza appena vista suggerisce la seguente definizione di albero binario di ricerca.

Definizione 6.1 (Albero binario di ricerca) *Un albero binario di ricerca è un albero binario che soddisfa le seguenti proprietà:*

- ① ogni nodo v contiene un elemento $elem(v)$ cui è associata una chiave $chiave(v)$ presa da un dominio totalmente ordinato;
- ② le chiavi nel sottoalbero sinistro di v sono $\leq chiave(v)$;
- ③ le chiavi nel sottoalbero destro di v sono $\geq chiave(v)$.

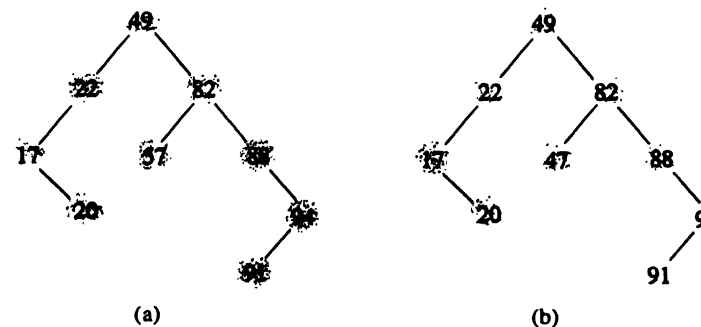


Figura 6.2 (a) Un albero binario di ricerca; (b) un albero binario non di ricerca. Per semplicità, mostriamo solo la chiave associata ad ogni nodo.

Le chiavi possono essere valori di qualunque tipo, purché su di esse sia definita una relazione d'ordine totale. I punti 2 e 3 nella Definizione 6.1 sono noti come *proprietà di ricerca*, e garantiscono che visitando un albero binario di ricerca in ordine simmetrico (come descritto nel Capitolo 3) si ottengano le chiavi in ordine non decrescente. Un esempio di albero binario di ricerca è mostrato in Figura 6.2(a). Il nodo con chiave 47 nell'albero in Figura 6.2(b), invece, non soddisfa la proprietà di ricerca rispetto alla radice.

Le idee di alto livello di una realizzazione di un dizionario tramite alberi binari di ricerca sono descritte in Figura 6.3. Discuteremo ora in maggior dettaglio come le singole operazioni possano essere implementate. Presenteremo anche la realizzazione di due operazioni per il calcolo del massimo di un sottoalbero e del predecessore di un nodo, utili per implementare l'operazione delete.

search(*chiave k*) $\rightarrow$ *elem*

Grazie alla proprietà di ricerca, implementare l'operazione search è molto semplice. Confrontiamo la chiave x che stiamo cercando con la chiave $chiave(v)$ della radice v dell'albero di ricerca. Se sono uguali, abbiamo trovato l'elemento. Altrimenti, proseguiamo la ricerca nel sottoalbero sinistro se $x < chiave(v)$ o in quello destro se $x > chiave(v)$. Ripetiamo la ricerca a partire dal figlio sinistro o destro della radice usando questa stessa strategia, finché troviamo x oppure arriviamo ad un albero vuoto, nel qual caso $x \notin S$. Lo pseudocodice dell'algoritmo search è riportato in Figura 6.4. Poiché dopo ogni confronto, se non abbiamo trovato l'elemento, scendiamo di un livello nell'albero T , il tempo richiesto dalla ricerca è $O(h)$, dove h è l'altezza di T . Osserviamo che se l'albero è molto profondo e sbilanciato, nel caso peggiore una ricerca costerà tempo $O(n)$, il che equivale ad usare una lista o un array disordinato.

Poiché l'approccio utilizzato dalla procedura search ricorda da vicino la ricerca binaria descritta nel Paragrafo 2.4.3 del Capitolo 2, è naturale chiedersi perché il tempo non sia $O(\log n)$. Osserviamo innanzitutto che gli insiemi di

classe AlberoBinarioDiRicerca **implementa** Dizionario:

dati: $S(n) = O(n)$
 un albero binario di ricerca T di altezza h e con n nodi, ciascuno contenente coppie ($elem$, $chiave$) in cui le chiavi sono prese da un universo totalmente ordinato.

operazioni:

$search(chiave\ k) \rightarrow elem$ $T(h) = O(h)$
 partendo dalla radice, traccia un cammino nell'albero per cercare un elemento con chiave k . Su ogni nodo, usa la proprietà di ricerca per decidere se proseguire nel sottoalbero sinistro o destro.

$insert(elem\ e, chiave\ k)$ $T(h) = O(h)$
 Crea un nuovo nodo v contenente la coppia (e , k), e lo aggiunge all'albero come foglia nella posizione opportuna, in modo da mantenere la proprietà di ricerca.

$delete(elem\ e)$ $T(h) = O(h)$
 se il nodo v contenente l'elemento e ha al più un figlio, elimina v collegando il figlio all'eventuale padre. Altrimenti scambia il nodo v con il suo predecessore ed elimina il predecessore.

Figura 6.3 Dizionario realizzato mediante alberi binari di ricerca.

valori $\geq X[n/2]$ e $\leq X[n/2]$ hanno cardinalità bilanciate, pari a circa $n/2$, e ricordiamo come è stato costruito l'albero T che abbiamo immaginato di associare all'array ordinato X : T ha un nodo al livello 0 (la radice), 2 nodi al livello 1, $2^2 = 4$ nodi al livello 2, e via dicendo. Sia h la profondità dell'ultimo livello, che contiene al più 2^h nodi. Poiché deve risultare $2^h \leq n$, si ottiene $h = O(\log n)$. Il numero di confronti eseguiti per effettuare una qualsiasi ricerca sull'albero T così costruito è quindi $O(\log n)$ nel caso peggiore, poiché l'altezza di T è limitata. Una tale limitazione non è però vera per un qualunque albero binario di ricerca.

$insert(elem\ e, chiave\ k)$

Un nuovo nodo con elemento e e chiave k viene sempre inserito come foglia dell'albero di ricerca. L'operazione $insert$ può quindi essere implementata in due passi:

Passo 1. Cerca il nodo v che diventerà genitore del nuovo nodo.

Passo 2. Crea un nuovo nodo u con elemento e e chiave k ed appendilo come figlio sinistro o destro di v rispettando la proprietà di ricerca.

Il Passo 1 equivale ad effettuare una ricerca della chiave k nell'albero, e richiede quindi tempo $O(h)$. Rappresentando l'albero tramite puntatori ai figli come descritto nel Paragrafo 3.3 del Capitolo 3, il Passo 2 richiede di modificare solo un numero costante di puntatori e può essere implementato in tempo $O(1)$. In totale, anche il costo dell'inserimento è quindi $O(h)$.

algoritmo $search(chiave\ k) \rightarrow elem$

1. $v \leftarrow$ radice di T
2. **while** ($v \neq null$) **do**
3. **if** ($k = chiave(v)$) **then return** $elem(v)$
4. **else if** ($k < chiave(v)$) **then** $v \leftarrow$ figlio sinistro di v
5. **else** $v \leftarrow$ figlio destro di v
6. **return null**

Figura 6.4 Implementazione iterativa dell'operazione $search$ in un albero binario di ricerca.

$max(nodo\ u) \rightarrow nodo$

Grazie alla proprietà di ricerca, per trovare il massimo di un sottoalbero di T radicato in un qualunque nodo u basta scendere verso destra nell'albero finché possibile, partendo da u , come mostrato in Figura 6.5. Il tempo di esecuzione è chiaramente $O(h)$.

$pred(nodo\ u) \rightarrow nodo$

Il predecessore di un nodo u è un nodo v avente massima chiave $\leq chiave(u)$. Per trovare il predecessore di u distinguiamo due casi:

1. u ha un figlio sinistro: in tal caso $pred(u)$ è il massimo del sottoalbero sinistro di u ;
2. u non ha un figlio sinistro: $pred(u)$, se esiste, è il più basso antenato di u (ovvero, l'antenato di u con massima profondità nell'albero) il cui figlio destro è anch'esso antenato di u . Per trovarlo, risaliamo da u verso la radice fino ad incontrare la prima "svolta a sinistra", come mostrato in Figura 6.7.

Lo pseudocodice dell'algoritmo $pred$ per la ricerca del predecessore è mostrato in Figura 6.6. Il tempo di esecuzione è chiaramente $O(h)$.

$delete(elem\ e)$

Come avviene in molte strutture dati, cancellare è più difficile di inserire. In

algoritmo $max(nodo\ u) \rightarrow nodo$

1. $v \leftarrow u$
2. **while** (figlio destro di $v \neq null$) **do**
3. $v \leftarrow$ figlio destro di v
4. **return** v

Figura 6.5 Ricerca del nodo con valore massimo nel sottoalbero dell'albero binario di ricerca T radicato nel nodo u .

```

algoritmo pred(nodo  $u$ )  $\rightarrow$  nodo
1.  if ( $u$  ha figlio sinistro  $\sin(u)$ ) then
2.    return max( $\sin(u)$ )
3.  while ( $\text{parent}(u) \neq \text{null}$  e  $u$  è figlio sinistro di suo padre) do
4.     $u \leftarrow \text{parent}(u)$ 
5.  return  $\text{parent}(u)$ 

```

Figura 6.6 Ricerca del predecessore di un nodo in un albero binario di ricerca.

particolare, per implementare l'operazione delete useremo la procedura pred per la ricerca del predecessore di un nodo. Sia u il nodo contenente l'elemento da cancellare. Distinguiamo tre casi.

1. u è una foglia: in tal caso basta distaccare la foglia dal genitore ed eliminarla;
2. u ha un unico figlio: sia v l'unico figlio di u . Se u è radice, v diviene la nuova radice dell'albero. Altrimenti, dopo aver individuato il genitore w di u , l'arco (w, u) viene sostituito dall'arco (w, v) come mostrato in Figura 6.8;
3. u ha due figli: ci si riconduce ad uno dei casi precedenti operando come segue. Si individua il predecessore di u , diciamo v : tale predecessore è certamente il massimo del sottoalbero sinistro di u , poiché u ha due figli. Osserviamo che v non può avere due figli, altrimenti non sarebbe predecessore. Dopo aver copiato $\text{chiave}(v)$ in $\text{chiave}(u)$, è quindi possibile cancellare fisicamente v dall'albero applicando uno dei due casi precedenti. La Figura 6.9 mostra un esempio.

È facile verificare che la proprietà di ricerca è sempre mantenuta. Inoltre, poiché la cancellazione di un nodo interno richiede l'individuazione del nodo da cancellare nonché del suo predecessore, il costo della cancellazione è $O(h)$.

Il seguente teorema riassume le prestazioni degli alberi binari di ricerca:

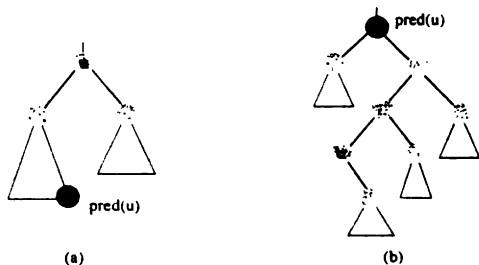


Figura 6.7 I due casi nella ricerca del predecessore del nodo u : (a) u ha un figlio sinistro; (b) u non ha un figlio sinistro.

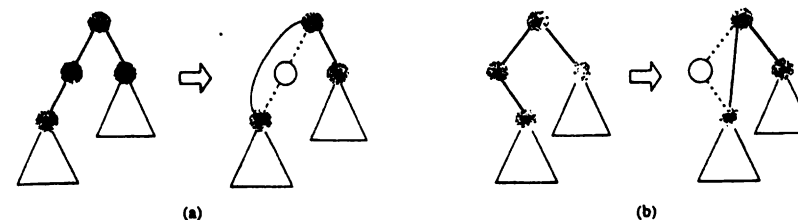


Figura 6.6 Cancellazione di un nodo con un unico figlio da un albero binario di ricerca.

Teorema 6.1 *Alberi binari di ricerca di altezza h sono in grado di supportare operazioni search, insert e delete in tempo $O(h)$.*

Abbiamo già osservato che nel caso peggiore l'altezza dell'albero può essere proporzionale al numero n di nodi. Se l'albero fosse bilanciato, però, avrebbe altezza logaritmica, e quindi potremmo sperare di implementare le varie operazioni in tempo $O(\log n)$. Nel prossimo paragrafo vedremo una definizione formale di bilanciamento e presenteremo una tecnica basata su rotazioni di sottoalberi utile per mantenere il bilanciamento a fronte di cambiamenti dinamici dell'insieme S .

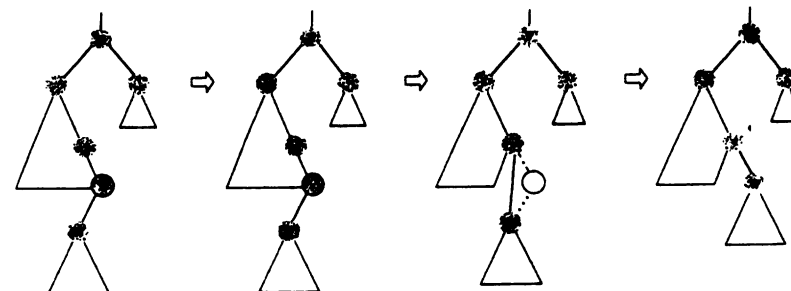


Figura 6.9 Cancellazione di un nodo con due figli da un albero binario di ricerca.

6.2 Alberi AVL

Dal Teorema 6.1 si evince che alberi di ricerca bilanciati garantiscono un tempo di ricerca logaritmico. Pur partendo da un albero bilanciato, però, a fronte di inserimenti e cancellazioni il bilanciamento dell'albero potrebbe perdersi, e quindi con l'aumentare del numero di operazioni effettuate le prestazioni del dizionario degraderebbero. È quindi opportuno mantenere il bilanciamento anche quando si inseriscono e si cancellano elementi. In questo paragrafo vedremo come ciò sia possibile. Innanzitutto, sottolineiamo che esistono varie definizioni formali di

bilanciamento, che legano ad esempio l'altezza dell'albero con il numero di nodi o le altezze dei suoi sottoalberi. Useremo qui l'idea di bilanciamento in altezza definita come segue.

Definizione 6.2 (Bilanciamento in altezza) Un albero è bilanciato in altezza se le altezze dei sottoalberi sinistro e destro di ogni nodo differiscono di al più un'unità.

Gli alberi binari di ricerca bilanciati in altezza sono anche detti *alberi AVL*, dai nomi degli ideatori Adel'son-Vel'skii e Landis, che li hanno introdotti nel 1962 [1]. In un albero AVL, oltre all'elemento e alla chiave, ciascun nodo mantiene anche un'informazione sul bilanciamento così definita.

Definizione 6.3 (Fattore di bilanciamento) Il fattore di bilanciamento $\beta(v)$ di un nodo v è la differenza tra l'altezza del sottoalbero sinistro e quella del sottoalbero destro di v .

$$\beta(v) = \text{altezza}(\text{sin}(v)) - \text{altezza}(\text{des}(v))$$

Il fattore di bilanciamento è tanto migliore quanto più basso è il suo valore assoluto, mentre è uguale all'altezza dell'albero quando questo degenera in lista. In particolare, in un albero binario completo, il fattore di bilanciamento è 0 su ogni nodo. Un albero binario di ricerca è quindi un albero AVL, in base alle Definizioni 6.2 e Definizioni 6.3, se il valore assoluto del fattore di bilanciamento è ≤ 1 su ogni nodo.

6.2.1 Altezza di un albero AVL

Dato lo stretto vincolo sui fattori di bilanciamento dei nodi, è lecito aspettarsi che l'altezza di un albero AVL con n nodi sia $O(\log n)$. Ed infatti ciò è vero, anche se dimostrarlo non è così immediato. Per farlo, tra tutti i possibili alberi bilanciati in altezza, studieremo l'altezza di quelli più sbilanciati possibile.

Definizione 6.4 Tra tutti gli alberi di altezza h bilanciati in altezza, un albero di Fibonacci ha il minimo numero di nodi.

Ben presto scopriremo da cosa derivi il nome *albero di Fibonacci*. Osserviamo che un albero di Fibonacci di altezza h può essere costruito unendo, tramite l'aggiunta di una radice, un albero di Fibonacci di altezza $h-1$ (ad esempio come sottoalbero sinistro) ed un albero di Fibonacci di altezza $h-2$ (ad esempio come sottoalbero destro), come mostrato in Figura 6.10. La figura mostra anche alcuni esempi di alberi di Fibonacci di altezza da 0 a 4.

È facile verificare che il fattore di bilanciamento di ogni nodo interno di un albero di Fibonacci costruito come in Figura 6.10 è $+1$, e quindi questi alberi sono gli alberi bilanciati in altezza più vicini alla condizione di non bilanciamento. Studieremo ora il rapporto tra l'altezza ed il numero di nodi di un albero di Fibonacci.

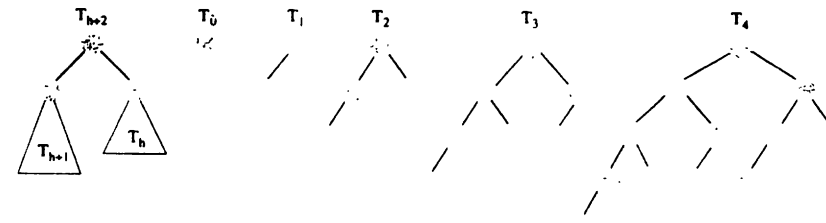


Figura 6.10 Costruzione ricorsiva di un albero di Fibonacci di altezza h ed esempi di alberi di Fibonacci di altezza da 0 a 4.

Lemma 6.1 Sia T_h un albero di Fibonacci di altezza h e sia n_h il numero dei suoi nodi. Risulta $h = \Theta(\log n_h)$.

Dimostrazione. Per costruzione di T_h , come mostrato in Figura 6.10, risulta $n_h = 1 + n_{h-1} + n_{h-2}$. Questo ricorda molto da vicino l'equazione dei numeri di Fibonacci: $F_i = F_{i-1} + F_{i-2}$ che abbiamo già incontrato nel Capitolo 1. E infatti dimostreremo per induzione che

$$n_h = F_{h+3} - 1$$

Il passo base, per $h = 0$, è banalmente verificato, essendo $n_0 = 1 = F_3 - 1 = 2 - 1$. Assumendo per ipotesi induttivamente che $n_k = F_{k+3} - 1$ per ogni $k < h$, ed usando le ricorrenze relative ad n_h ed a F_i , si ha:

$$n_h = 1 + n_{h-1} + n_{h-2} = 1 + F_{h+2} - 1 + F_{h+1} - 1 = F_{h+3} - 1$$

Ricordiamo ora dal Capitolo 1 che $F_h = \Theta(\phi^h)$, dove $\phi \approx 1.618$ è la sezione aurea (per una stima precisa di F_n in termini della costante ϕ rimandiamo all'Equazione 17.13 riportata in Appendice). L'altezza ed il numero di nodi di T_h sono quindi esponenzialmente correlate, e pertanto $h = \Theta(\log n_h)$. $\square$

Adel'son-Vel'skii e Landis hanno dimostrato delle stime ancor più precise sull'altezza di un albero di Fibonacci con n_h nodi. La stima asintotica data nel Lemma 6.1 ci permette comunque di concludere che un qualunque albero AVL ha altezza logaritmica nel numero di nodi, come dimostrato nel seguente corollario.

Corollario 6.1 Un albero AVL con n nodi ha altezza $O(\log n)$.

Dimostrazione. Sia h l'altezza dell'albero AVL. Per dimostrare che $h = O(\log n)$, consideriamo l'albero di Fibonacci di altezza h . Sia n_h il numero di nodi di tale albero. Per definizione di albero di Fibonacci, $n_h \leq n$. Il fatto che $h = O(\log n_h)$ in base al Lemma 6.1 completa la dimostrazione. $\square$

Il Corollario 6.1 implica che la ricerca in un albero AVL, effettuata tramite lo stesso algoritmo search utilizzato per alberi binari di ricerca qualunque, ha costo $O(\log n)$. Dobbiamo però ancora vedere come mantenere il bilanciamento a fronte di inserimenti e cancellazioni: questo sarà l'argomento dei prossimi paragrafi.

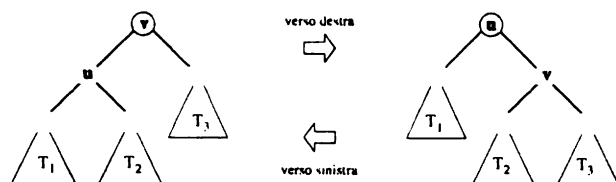


Figura 6.11 Rotazioni di base: le rotazioni verso sinistra e verso destra sono simmetriche. Il nodo v è il perno della rotazione verso destra, mentre il nodo u è il perno della rotazione verso sinistra.

6.2.2 Ribilanciamento tramite rotazioni

Poiché inserimenti e cancellazioni potrebbero far perdere il bilanciamento in un albero AVL, dovremo ripristinarlo usando opportune rotazioni. La Figura 6.11 mostra la rotazione di base: si può ruotare su un nodo perno o verso destra o verso sinistra, e la figura mostra che i due casi sono perfettamente simmetrici. La proprietà di ricerca è mantenuta dopo la rotazione: infatti l'ordine relativo delle chiavi nei sottoalberi T_1 , T_2 e T_3 e nei nodi u e v rimane invariato. Una rotazione di base è anche detta rotazione *semplice*. Componendo opportunamente due rotazioni di base otteniamo una rotazione *doppia*.

Come vedremo, le rotazioni vengono effettuate su nodi sbilanciati, ovvero su nodi il cui fattore di bilanciamento in valore assoluto è ≥ 2 . Sia v un tale nodo con fattore di bilanciamento ± 2 . Intuitivamente, esisterà un sottoalbero di v che è "troppo alto", e che quindi sbilancia il nodo. A seconda della posizione di questo sottoalbero, che chiameremo T , possiamo avere quattro casi:

Sinistra - sinistra	(SS)	T è il sottoalbero sinistro del figlio sinistro di v
Destra - destra	(DD)	T è il sottoalbero destro del figlio destro di v
Sinistra - destra	(SD)	T è il sottoalbero destro del figlio sinistro di v
Destra - sinistra	(DS)	T è il sottoalbero sinistro del figlio destro di v

Poiché questi casi sono simmetrici a coppie (SS con DD, e SD con DS), nel seguito vedremo solo come trattare SS e SD.

SS: per ribilanciare il nodo v basterà applicare una rotazione semplice verso destra su v , il cui fattore di bilanciamento passerà da 2 a 0. Si noti che l'altezza del sottoalbero coinvolto nella rotazione è $h + 3$ prima della rotazione (per via del sottoalbero T_1), ed è $h + 2$ immediatamente dopo.

SD: questo caso è più complesso e richiede l'applicazione di una rotazione doppia. Sia z il figlio sinistro di v : l'albero T che sbilancia v è il sottoalbero destro di z , radicato nel nodo w . La rotazione SD consiste in una rotazione di base verso sinistra con perno in z seguita da una rotazione di base verso destra con perno in v . La Figura 6.13 mostra l'effetto finale di tale rotazione doppia: la figura presenta due casi perché, a seconda del fatto che lo sbilanciamento sia causato dal sottoalbero sinistro o destro di w , i valori dei fattori di bilanciamento dei nodi coinvolti

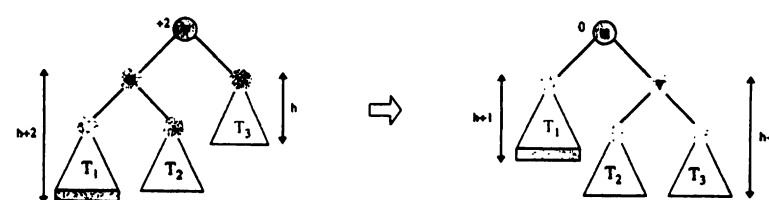


Figura 6.12 Rotazione SS: Il rettangolo evidenzia il sottoalbero che sbilancia il nodo v .

nella rotazione saranno diversi. Anche in questo caso l'altezza del sottoalbero coinvolto nella rotazione è $h + 3$ prima e diventa $h + 2$ dopo. Possiamo quindi enunciare la seguente proprietà, che ci sarà utile in seguito:

Proprietà 6.1 *Una rotazione SS, SD, DD, o DS applicata ad un nodo v con fattore di bilanciamento ± 2 fa decrescere di 1 l'altezza del sottoalbero radicato in v prima della rotazione.*

6.2.3 Modifiche del dizionario

In questo paragrafo mostreremo come usare le rotazioni per ripristinare il bilanciamento dell'albero AVL dopo un inserimento o una cancellazione. Sottolineiamo che ogni rotazione (sia semplice che doppia) richiede tempo $O(1)$. Poiché è importante mantenere il tempo di esecuzione delle operazioni *insert* e *delete* proporzionale all'altezza dell'albero (come nel Teorema 6.1), cercheremo di limitare il numero di rotazioni eseguite a fronte di ciascuna operazione.

`insert(elem  $e$ , chiave  $k$ )`

L'inserimento avviene in tre passi:

Passo 1. Si crea un nuovo nodo u con elemento e e chiave k e lo si inserisce nell'albero come descritto nel Paragrafo 6.1: il nodo diviene quindi una foglia.

Passo 2. Si ricalcolano i fattori di bilanciamento che sono mutati in seguito all'inserimento. Sia r la radice dell'albero AVL. Osserviamo che solo i fattori di bilanciamento dei nodi nel cammino da r ad u possono mutare, e questi possono essere facilmente ricalcolati risalendo nel cammino dalla foglia verso la radice.

Passo 3. Se nel cammino da r ad u appare un fattore di bilanciamento pari a ± 2 , occorre ribilanciare tramite rotazioni.

Dettagliamo ora meglio il Passo 3. Sia v il più profondo nodo con fattore di bilanciamento pari a ± 2 , che chiameremo *nodo critico*: il ribilanciamento consiste nell'eseguire una rotazione con perno in v , secondo i casi descritti nel Paragrafo 6.2.2. Dimostriamo ora la correttezza di questo procedimento:

Lemma 6.2 *Una rotazione semplice o doppia sul nodo critico è sufficiente a ribilanciare in altezza un albero AVL dopo l'inserimento di un nuovo elemento.*

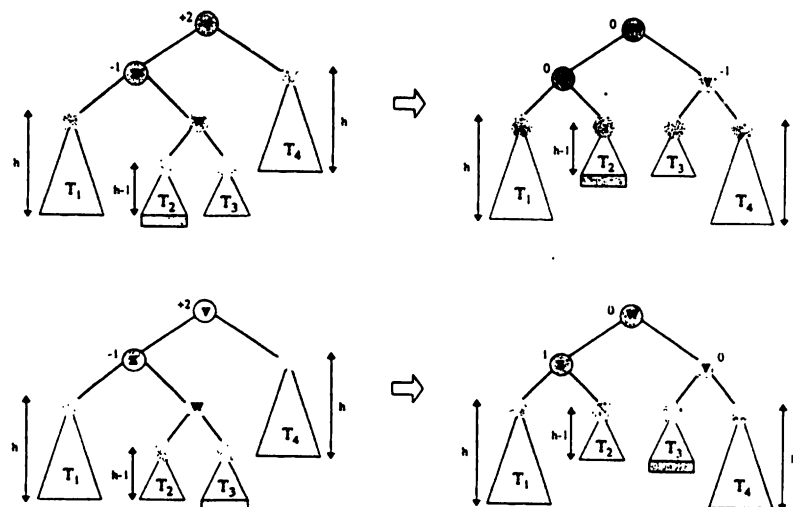


Figura 6.13 Rotazione SD: i due casi sono trattati analogamente, ma mostrano che i fattori di bilanciamento dei nodi coinvolti nella rotazione sono diversi.

Dimostrazione. Sia T_v l'albero radicato nel nodo critico v : T_v si sbilancia perché abbiamo inserito un nodo in un suo sottoalbero la cui altezza è aumentata di 1. Chiamiamo h_1 l'altezza di T_v prima dell'inserimento; h_2 l'altezza di T_v dopo l'inserimento ma prima della rotazione; ed h_3 l'altezza di T_v dopo la rotazione. Risulta $h_2 = h_1 + 1$. Inoltre, per la Proprietà 6.1 si ha $h_3 = h_2 - 1 = h_1$. In totale, l'altezza di T_v non varia, e quindi nessun antenato vede il proprio fattore di bilanciamento mutare. $\square$

Il Lemma 6.2 implica che una volta identificato il nodo critico ed applicata l'opportuna rotazione, non occorre risalire nell'albero ed effettuare altre rotazioni, perché i fattori di bilanciamento più in alto non variano. Ricordiamo dal Corollario 6.1 che l'altezza dell'albero è logaritmica nel numero di nodi. Il costo di un inserimento è pertanto $O(\log n)$, essendo dato dalla somma del costo $O(\log n)$ per i Passi 1 e 2, e del costo $O(1)$ per il Passo 3.

`delete(elem e)`

Anche la cancellazione avviene in tre passi:

Passo 1. Si cancella il nodo come descritto nel Paragrafo 6.1.

Passo 2. Si ricalcolano i fattori di bilanciamento che sono mutati in seguito alla cancellazione. Osserviamo che solo i fattori di bilanciamento dei nodi nel cammino dalla radice al padre del nodo eliminato possono mutare, e questi possono essere ricalcolati risalendo nell'albero dal basso verso l'alto.

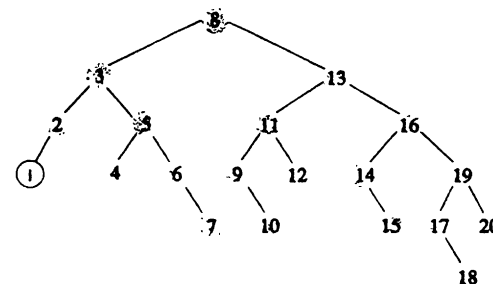


Figura 6.14 La cancellazione del nodo con chiave 1 genera una serie di rotazioni a cascata.

Passo 3. Per ogni nodo con fattore di bilanciamento pari a ± 2 , procedendo dal basso verso l'alto, si opera una rotazione semplice o doppia.

Osserviamo che, diversamente dall'inserimento, nel Passo 3 potremmo eseguire $O(\log n)$ rotazioni nel caso peggiore. Sia infatti T_v l'albero radicato nel nodo critico v . Sicuramente T_v si sbilancia perché abbiamo cancellato un nodo da un suo sottoalbero la cui altezza diminuisce di 1. Chiamiamo h_1 l'altezza di T_v prima dell'eliminazione del nodo; h_2 l'altezza di T_v dopo l'eliminazione del nodo ma prima della rotazione; ed h_3 l'altezza di T_v dopo la rotazione. Diversamente dall'inserimento, risulta $h_1 = h_2$. Inoltre, per la Proprietà 6.1 si ha $h_3 = h_2 - 1 = h_1 - 1$. Quindi, in totale l'altezza di T_v diminuisce e qualche antenato di v con fattore di bilanciamento ± 1 potrebbe risentirne e sbilanciarsi a sua volta. Un esempio in cui lo sbilanciamento si propaga a cascata verso l'alto è mostrato in Figura 6.14: basta cancellare il nodo con chiave 1 ed osservare cosa accade.

Pur essendo un po' più complessa dell'inserimento, la cancellazione ha comunque lo stesso costo asintotico $O(\log n)$. Infatti potranno essere eseguite al più $O(\log n)$ rotazioni, tante quante l'altezza dell'albero, e ciascuna di esse richiede tempo $O(1)$. Esistono altri tipi di alberi binari di ricerca bilanciati, quali gli alberi *red-black*, in cui anche le cancellazioni possono essere implementate eseguendo un numero costante di rotazioni.

Mostriamo in Figura 6.15 la struttura di una possibile classe `AlberoAVL` derivata dalla classe base `AlberoBinarioDiRicerca`. L'unica informazione che ogni nodo dell'albero ha in più rispetto ai nodi di un albero binario di ricerca è il fattore di bilanciamento. L'operazione `search` è direttamente ereditata dalla classe base. Le operazioni di aggiornamento `insert` e `delete` sono invece realizzate richiamando prima le corrispondenti operazioni ereditate, che eseguono il grosso del lavoro, e poi ricalcolando i fattori di bilanciamento nel cammino critico e ribilanciando tramite opportune rotazioni. Riassumiamo le prestazioni degli alberi AVL nel seguente teorema:

classe `AlberoAVL` estende `AlberoBinarioDiRicerca`:

dati: $S(n) = O(n)$
 albero binario di ricerca T ereditato, più il fattore di bilanciamento di ogni nodo.

operazioni:
 $search(chiave\ k) \rightarrow elem$ $T(n) = O(\log n)$
 ereditata.

$insert(elem\ e, chiave\ k)$ $T(n) = O(\log n)$
 chiama `insert()` ereditata, poi ricalcola i fattori di bilanciamento ed eventualmente ribilancia tramite $O(1)$ rotazioni.

$delete(elem\ e)$ $T(n) = O(\log n)$
 chiama `delete()` ereditata, poi ricalcola i fattori di bilanciamento ed eventualmente ribilancia tramite $O(1)$ rotazioni.

Figura 6.15 Dizionario `AlberoAVL` come estensione della classe `AlberoBinarioDiRicerca` di Figura 6.3.

Teorema 6.2 *Alberi AVL con n nodi sono in grado di supportare operazioni `search`, `insert` e `delete` in tempo $O(\log n)$ nel caso peggiore.*

6.3 * Alberi auto-aggiustanti

Gli alberi AVL descritti nel Paragrafo 6.2 mantengono esplicitamente una condizione di bilanciamento, basata sul fattore di bilanciamento dei nodi, per ottenere un tempo di esecuzione logaritmico nel caso peggiore per tutte le operazioni. Se siamo disposti ad accettare tempi di esecuzione logaritmici ammortizzati su una sequenza di operazioni, non abbiamo bisogno di mantenere alcuna condizione di bilanciamento in maniera esplicita, come vedremo in questo paragrafo. Descriveremo infatti una struttura dati che gode di questa proprietà: gli alberi binari di ricerca auto-aggiustanti (o anche *splay trees*) dovuti a Sleator e Tarjan [10]. L'idea fondamentale degli *splay trees* è che l'albero viene automaticamente riaggiustato ogni volta che si esegue una qualunque operazione: in particolare, l'elemento cui si accede viene fatto risalire alla radice ed i sottoalberi incontrati lungo il cammino sono riposizionati tramite opportune rotazioni.

6.3.1 L'operazione splay

L'euristica di ristrutturazione proposta da Sleator e Tarjan è anche nota come operazione *splay*. Lo *splay* di un nodo u consiste nel partire da u e risalire fino alla radice dell'albero eseguendo una sequenza di rotazioni. Per brevità, denoteremo con $p(u)$ il padre di u nell'albero. Chiameremo inoltre *nonno* di u , e lo indicheremo

mo con $p^2(u)$, il padre di $p(u)$. In ciascun passo di *splay*, la rotazione da eseguire viene scelta come segue:

- se u è figlio della radice, ruotiamo su $p(u)$;
- se u ha un nonno, $p^2(u)$, ed u e $p(u)$ sono entrambi figli sinistri o destri, ruotiamo, prima su $p^2(u)$ e poi su $p(u)$;
- se u ha un nonno, $p^2(u)$, ed u è figlio sinistro mentre $p(u)$ è figlio destro o viceversa, ruotiamo su $p(u)$ e poi sul nuovo genitore di u (ovvero $p^2(u)$ prima della rotazione).

I vari casi sono illustrati in Figura 6.16. Ciascun passo di *splay* richiede tempo $O(1)$. L'effetto dell'euristica di ristrutturazione è di far risalire u alla radice, riarrangiando il resto del cammino. Per rendere un albero binario di ricerca auto-aggiustante, eseguiamo l'operazione *splay* in corrispondenza di ciascuna operazione di accesso o di modifica dell'albero. In particolare:

- dopo aver cercato una chiave, eseguiamo uno *splay* sul nodo che contiene la chiave o sulla foglia su cui la ricerca è terminata;
- dopo aver inserito un elemento, eseguiamo uno *splay* sul nuovo nodo che contiene l'elemento;
- dopo aver eliminato un nodo v (che, per quanto detto nel Paragrafo 6.1, potrebbe essere il predecessore dell'elemento di cui si richiedeva la cancellazione), eseguiamo uno *splay* sul genitore di v appena prima dell'eliminazione.

In ciascun caso, il tempo dell'operazione è proporzionale alla lunghezza del cammino su cui lo *splay* procede. Mostriamo in Figura 6.17 la struttura di una possibile classe `AlberoSplay` derivata dalla classe base `AlberoBinarioDiRicerca`. Osserviamo che non esistono informazioni aggiuntive a carico dei nodi o degli archi di T , come invece avveniva nel caso degli alberi AVL. Tutte le operazioni sono realizzate richiamando prima le corrispondenti operazioni ereditate, e poi eseguendo l'euristica *splay* su un nodo opportunamente scelto, a seconda dell'operazione. Il tempo ammortizzato delle operazioni su una sequenza è $O(\log n)$, come dimostreremo nel prossimo paragrafo. L'effetto dell'euristica *splay* è illustrato in Figura 6.18.

6.3.2 Analisi basata sul potenziale

Mostriamo ora che il costo ammortizzato di una operazione di *splay* è $O(\log n)$. Useremo il metodo del potenziale introdotto nel Paragrafo 2.7 del Capitolo 2.

Definizione 6.5 *Sia T uno splay tree. Per ogni nodo x , sia $d(x)$ il numero di discendenti di x , incluso se stesso. Definiamo rank di x il valore $r(x) = \log d(x)$ e la funzione potenziale come*

$$\Phi(T) = \sum_{x \in T} r(x)$$